Manual de Usuario

St4ck Over-Pig - Compilador a RISC-V

Compiladores 2 - USAC CUNOC

Brandon Gustavo Güinac Román - 201931217

Octubre 2026

1. Requisitos e inicio

- Java 17 o superior y Maven 3.9 (para compilar desde el código fuente).
- Compilar: mvn clean compile. Ejecutar la interfaz gráfica desde la clase principal del proyecto (Main).
- Opcional: gcc para probar el C generado, y RARS (rars.jar) como simulador externo. No son necesarios para ejecutar programas dentro de la aplicación.

2. La interfaz

La ventana tiene un editor de código con resaltado y, debajo, pestañas de resultados.

Elemento	Función
Compilar	Analiza el programa y genera cuartetas, C y RISC-V. Muestra errores léxicos, sintácticos y semánticos en la pestaña Reporte.
Compilar y ejecutar ▶️	Compila y ejecuta el ensamblador generado dentro de la aplicación, con el intérprete embebido.
Guardar cuartetas (.c3d)…	Guarda el código de tres direcciones.
Guardar código C…	Guarda el C generado (solo etiquetas y goto).
Guardar RISC-V (.asm)…	Guarda el ensamblador, listo para abrir en RARS.
Pestaña Reporte	Errores y resumen de la compilación, tabla de símbolos.
Pestaña Ejecución	Salida del programa, campo de entrada (Enter envía el texto) y botón Detener.
Pestañas Cuartetas / C / RISC-V	Código generado, solo lectura.

2.1 Árbol de archivos y proyecto

El panel izquierdo muestra la carpeta del proyecto como árbol de carpetas y archivos. Un clic abre el archivo en el editor.

Acción	Cómo
Abrir carpeta de proyecto	Archivo > Abrir carpeta de proyecto…
Abrir un archivo suelto	Archivo > Abrir archivo…
Crear archivo o carpeta	Archivo > Nuevo archivo / Nueva carpeta, o clic derecho en el árbol.
Guardar / Guardar como	Archivo > Guardar o Guardar como…
Descargar	Archivo > Descargar selección…, o clic derecho > Descargar. Un archivo se copia donde usted elija; una carpeta (o todo el proyecto si no hay selección) se descarga como .zip conservando sus subcarpetas.
Actualizar el árbol	Clic derecho > Actualizar

El editor resalta la sintaxis en tiempo real según la extensión del archivo (.y, .z o .pig).

2.2 Galería de funcionalidades

Capturas de la aplicación mostrando cada funcionalidad.

Figura 1. Lenguaje Y? con resaltado de sintaxis

Al abrir un archivo .y el editor colorea palabras reservadas, números, cadenas y comentarios en tiempo real. El panel izquierdo es el árbol del proyecto.

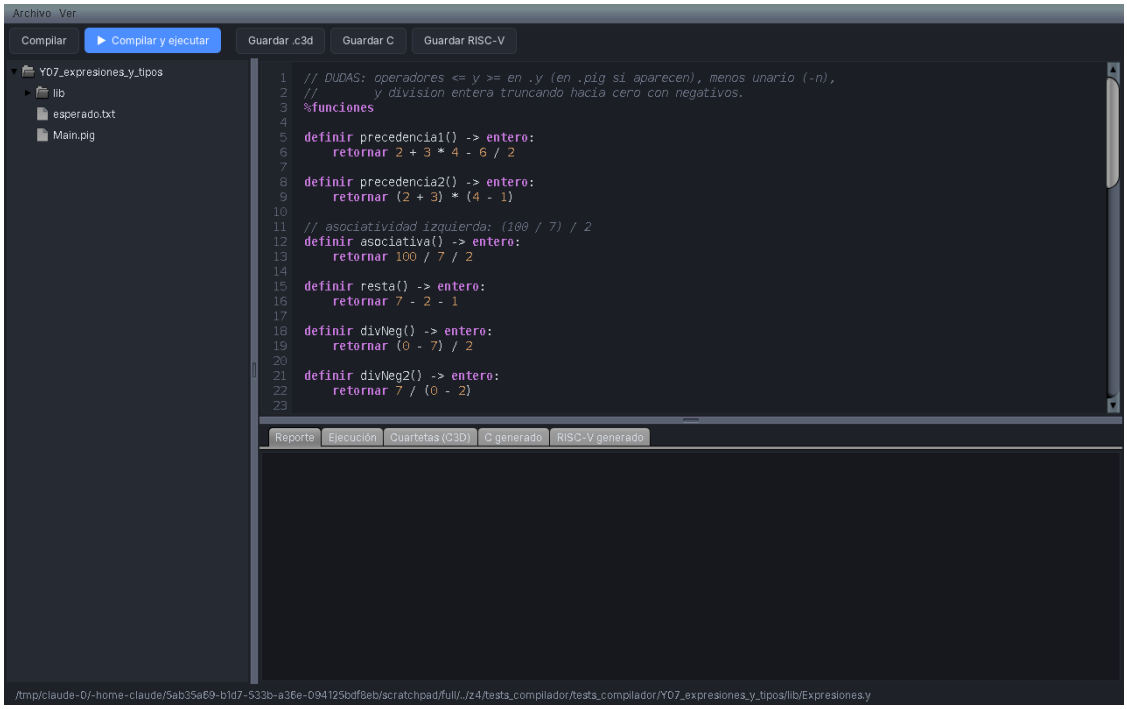


Figura 2. Compilación de un archivo Y?

El botón Compilar analiza el archivo y la pestaña Reporte resume errores y cuartetas generadas.

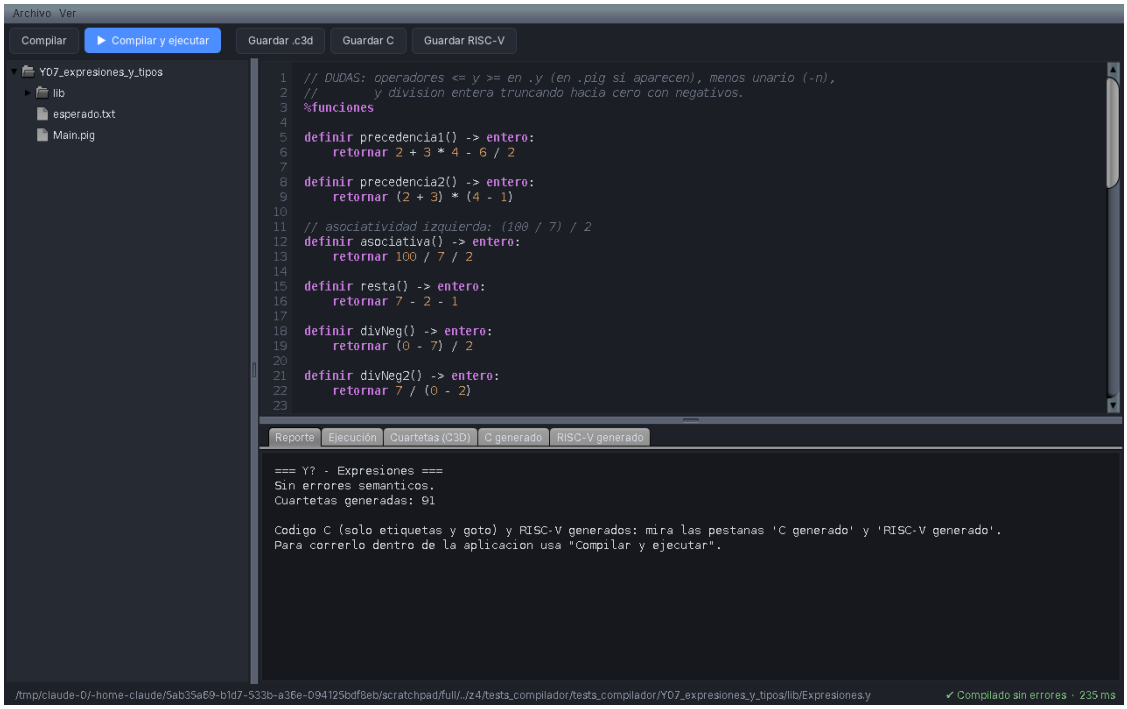


Figura 3. Lenguaje Zetariano

Clases con herencia (extends), super(...), this y @Override. Las palabras reservadas de Zetariano se resaltan igual en tiempo real.

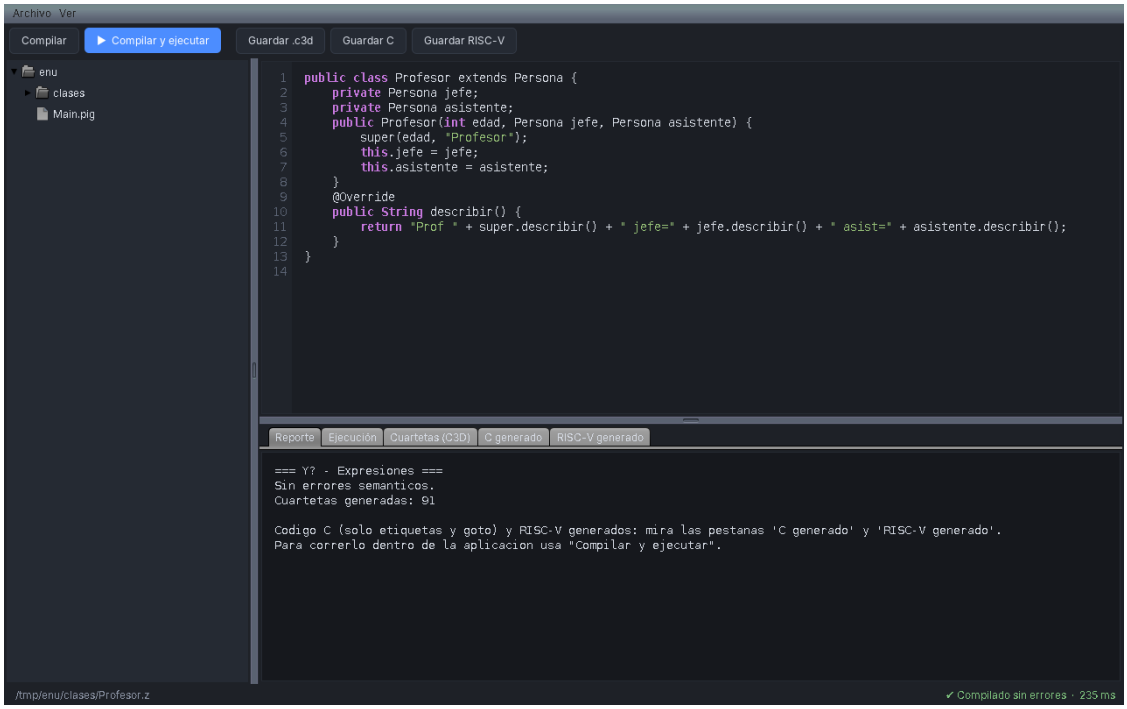


Figura 4. Programa Pig Latin con objetos

Pig Latin con imports de clases .z y declaración de objetos con tipo explícito: esto x : novus Persona(...) : Persona ; y objetos nulos.

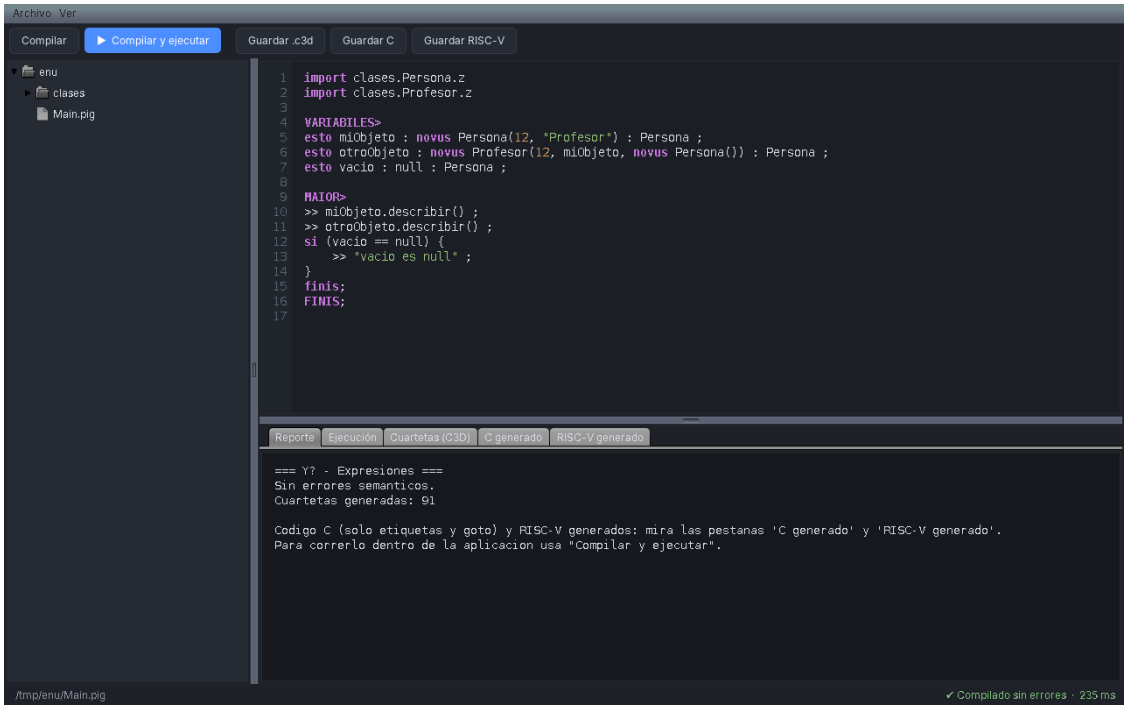


Figura 5. Informe de compilación de Pig Latin

Importaciones resueltas, errores semánticos (si los hay) y cantidad de cuartetas generadas.

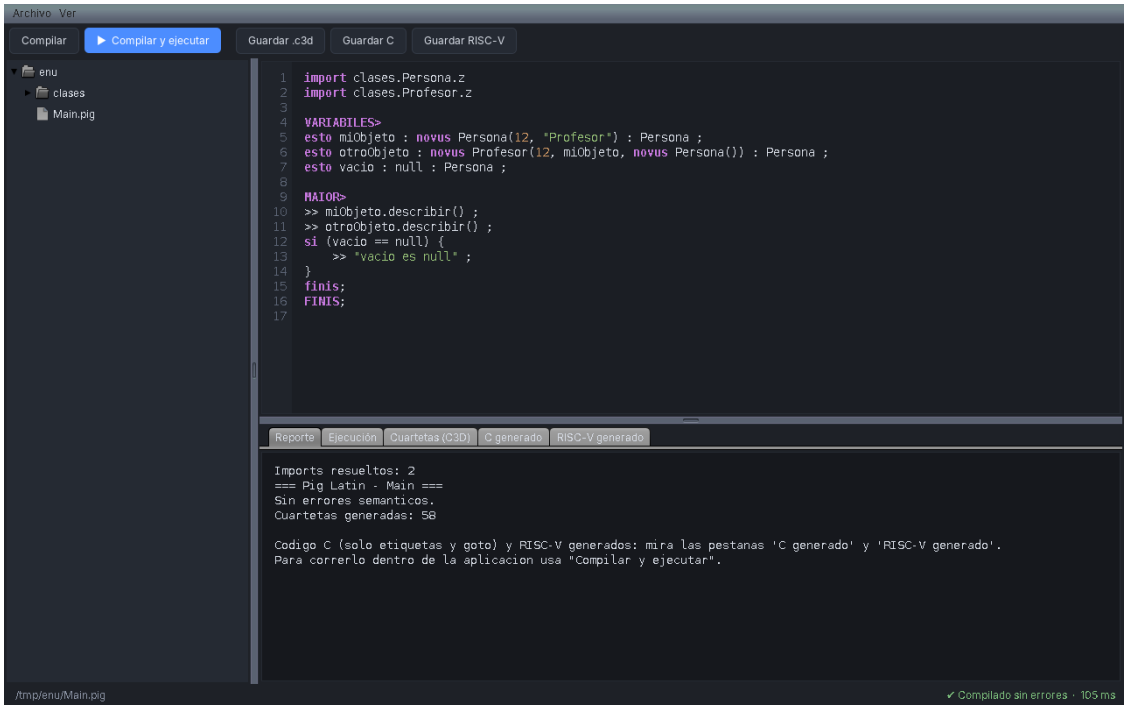


Figura 6. Código de tres direcciones (cuartetas)

La pestaña Cuartetas (C3D) muestra el código intermedio legible; el botón Guardar .c3d lo exporta.

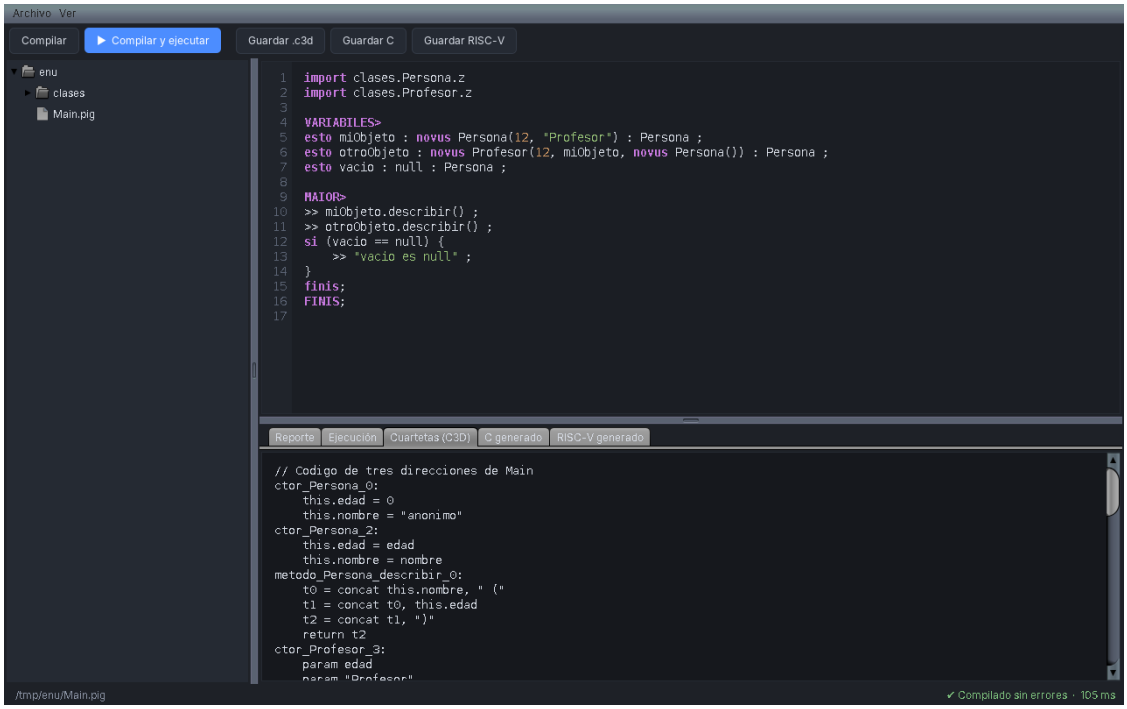


Figura 7. Código C con etiquetas y goto

La pestaña C generado contiene un único main que usa solo etiquetas y goto; Guardar C lo exporta.

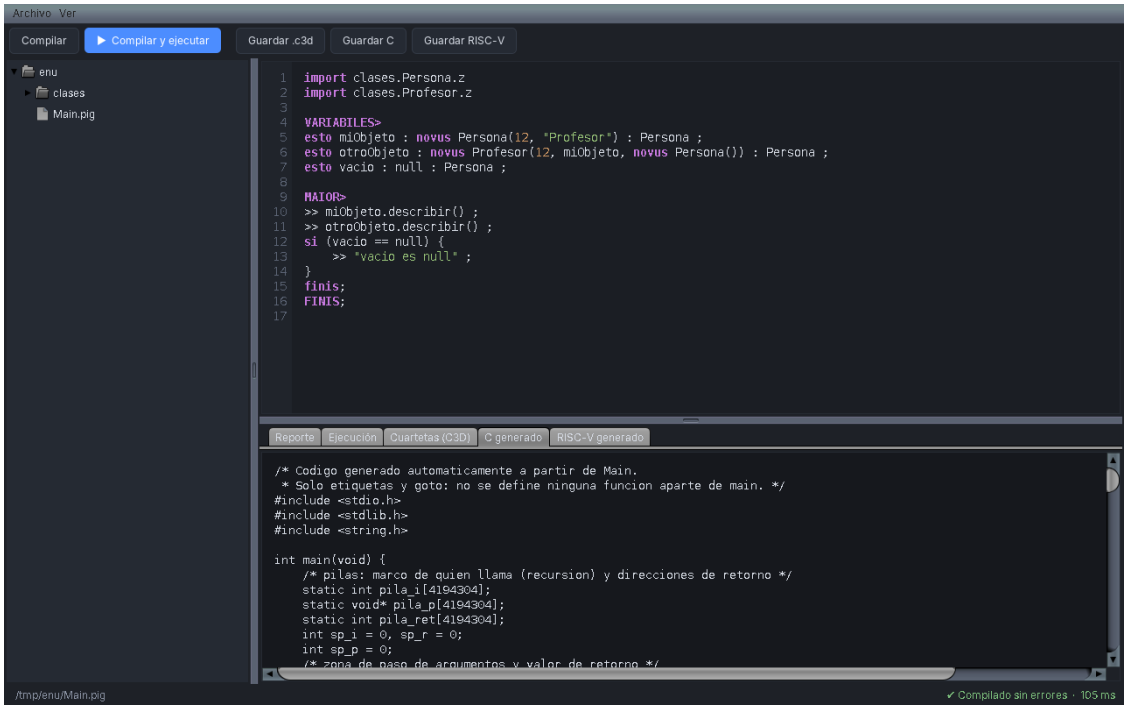


Figura 8. Ensamblador RISC-V

La pestaña RISC-V generado muestra el .asm compatible con RARS; Guardar RISC-V lo exporta.

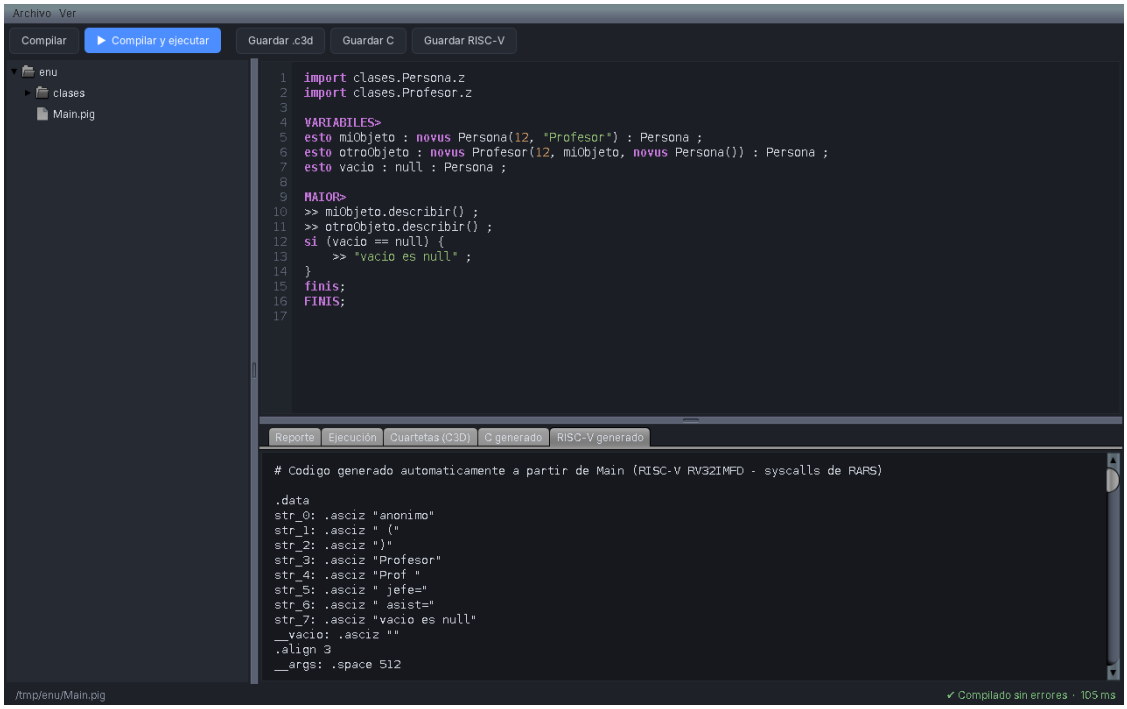


Figura 9. Ejecución dentro de la aplicación

Compilar y ejecutar corre el ensamblador con el intérprete RISC-V embebido. La salida aparece en la pestaña Ejecución.

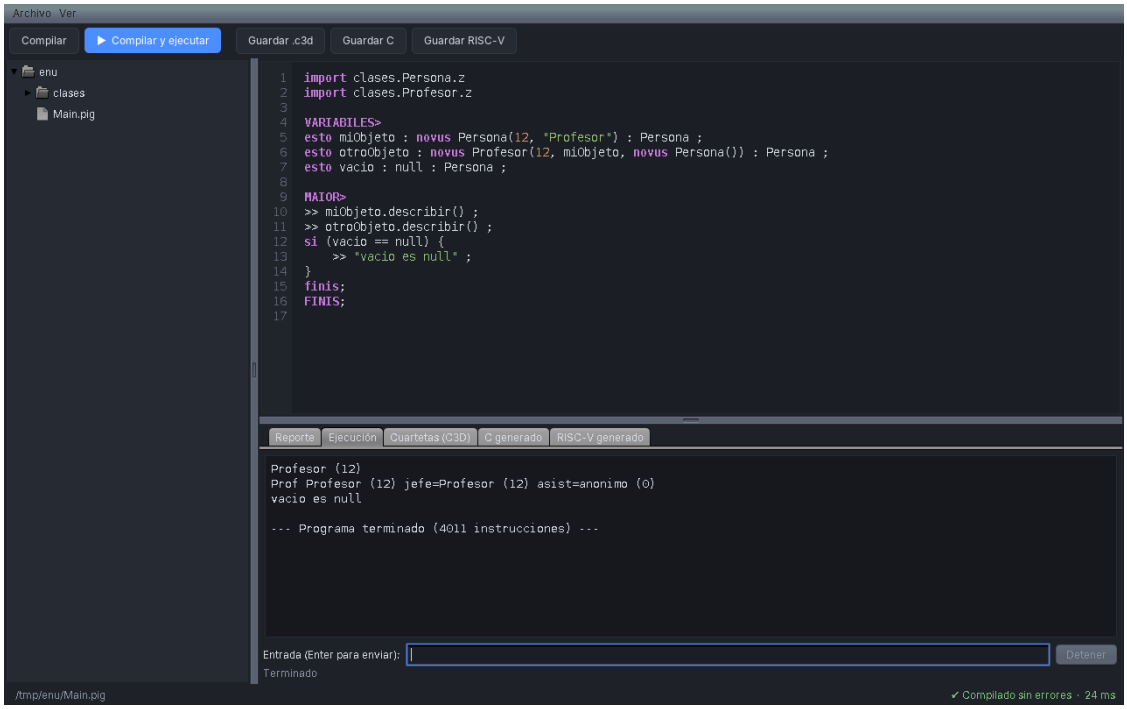


Figura 10. Entrada de datos del usuario

Cuando el programa lee datos, la pestaña Ejecución espera la entrada en el campo inferior (Enter la envía).

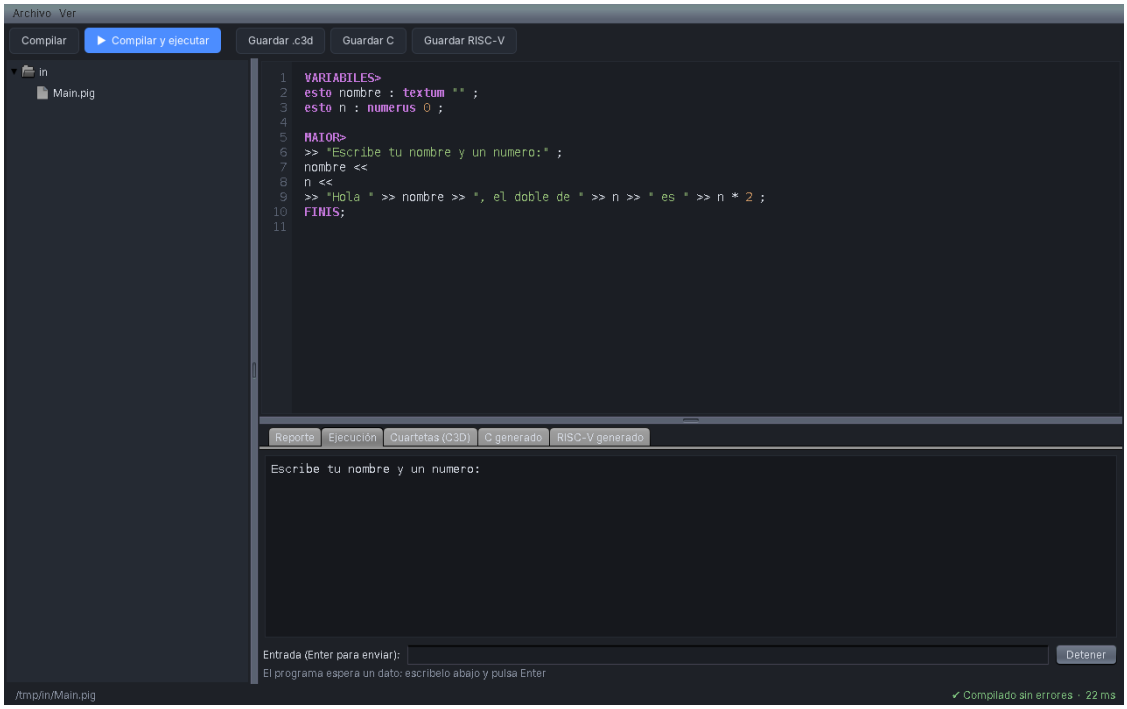
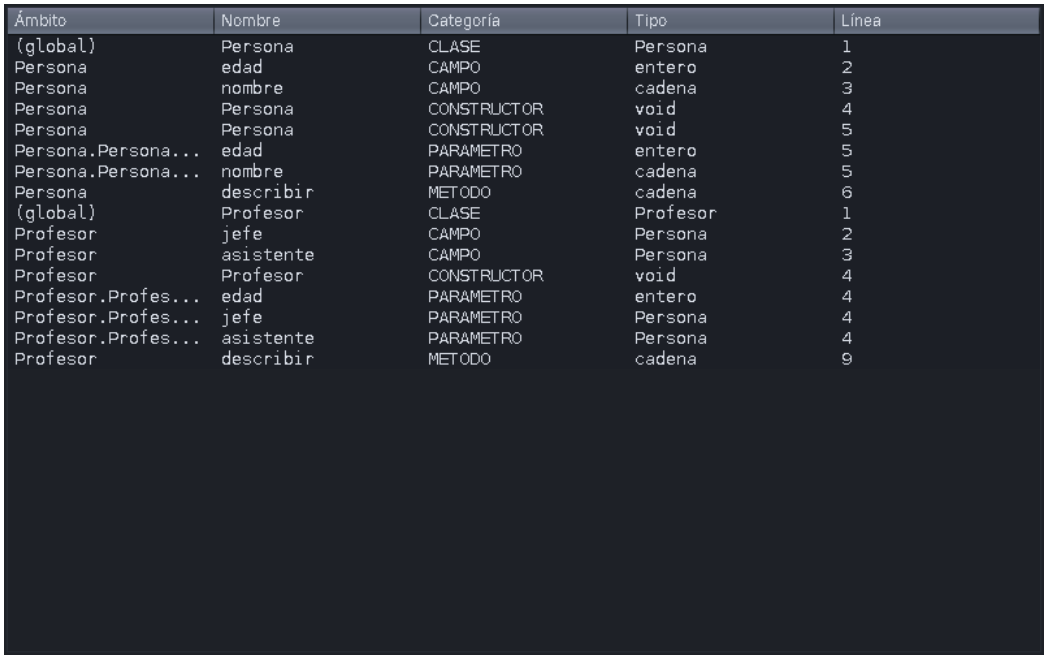


Figura 11. Tabla de símbolos

Menú Ver > Tabla de símbolos: ámbito, nombre, categoría, tipo y línea de cada símbolo declarado.



Ámbito	Nombre	Categoría	Tipo	Línea
(global)	Persona	CLASE	Persona	1
Persona	edad	CAMPO	entero	2
Persona	nombre	CAMPO	cadena	3
Persona	Persona	CONSTRUCTOR	void	4
Persona	Persona	CONSTRUCTOR	void	5
Persona.Persona...	edad	PARAMETRO	entero	5
Persona.Persona...	nombre	PARAMETRO	cadena	5
Persona	describir	METODO	cadena	6
(global)	Profesor	CLASE	Profesor	1
Profesor	jefe	CAMPO	Persona	2
Profesor	asistente	CAMPO	Persona	3
Profesor	Profesor	CONSTRUCTOR	void	4
Profesor.Profes...	edad	PARAMETRO	entero	4
Profesor.Profes...	jefe	PARAMETRO	Persona	4
Profesor.Profes...	asistente	PARAMETRO	Persona	4
Profesor	describir	METODO	cadena	9

Figura 12. Error léxico

Un carácter no reconocido (@) se reporta con línea y columna.

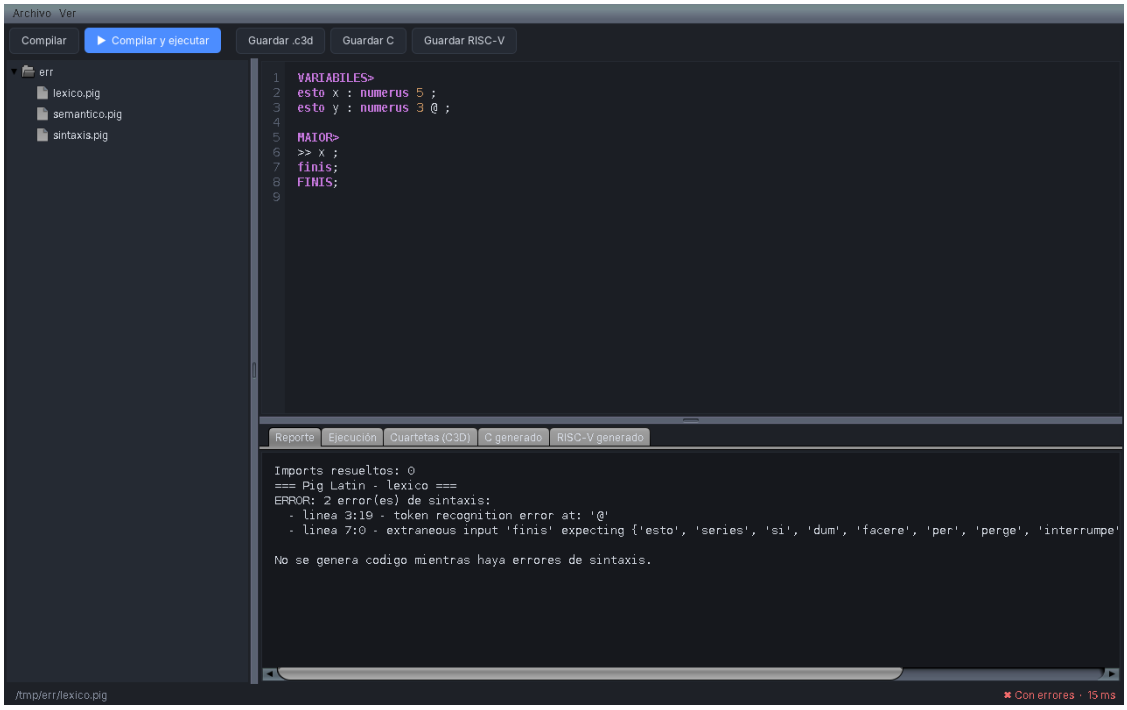


Figura 13. Error sintáctico

Un paréntesis sin cerrar se reporta con línea y columna; no se genera código.

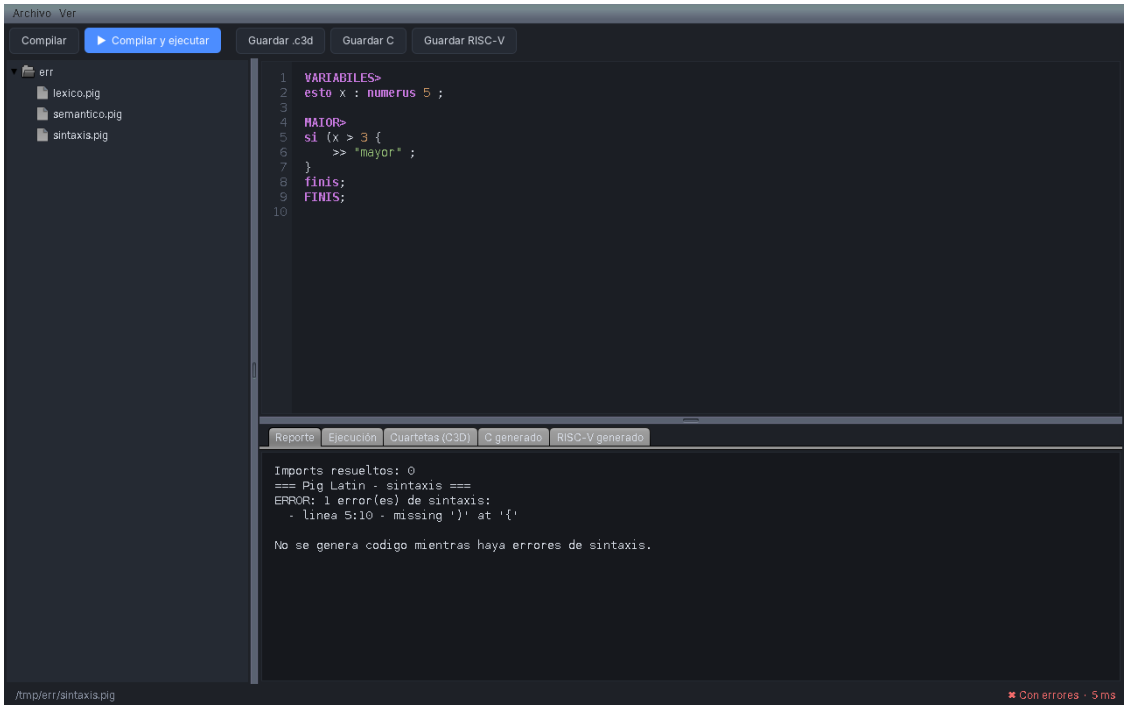


Figura 14. Error semántico

Asignar una cadena a un entero o usar una variable no declarada se reporta con su línea.

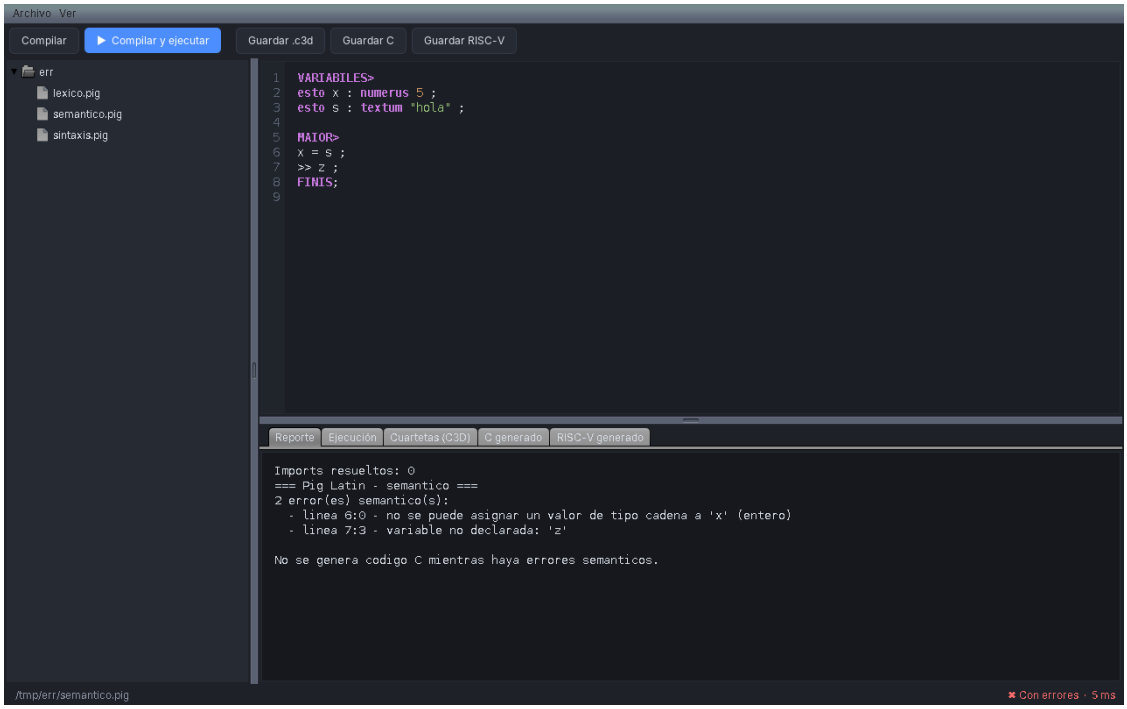
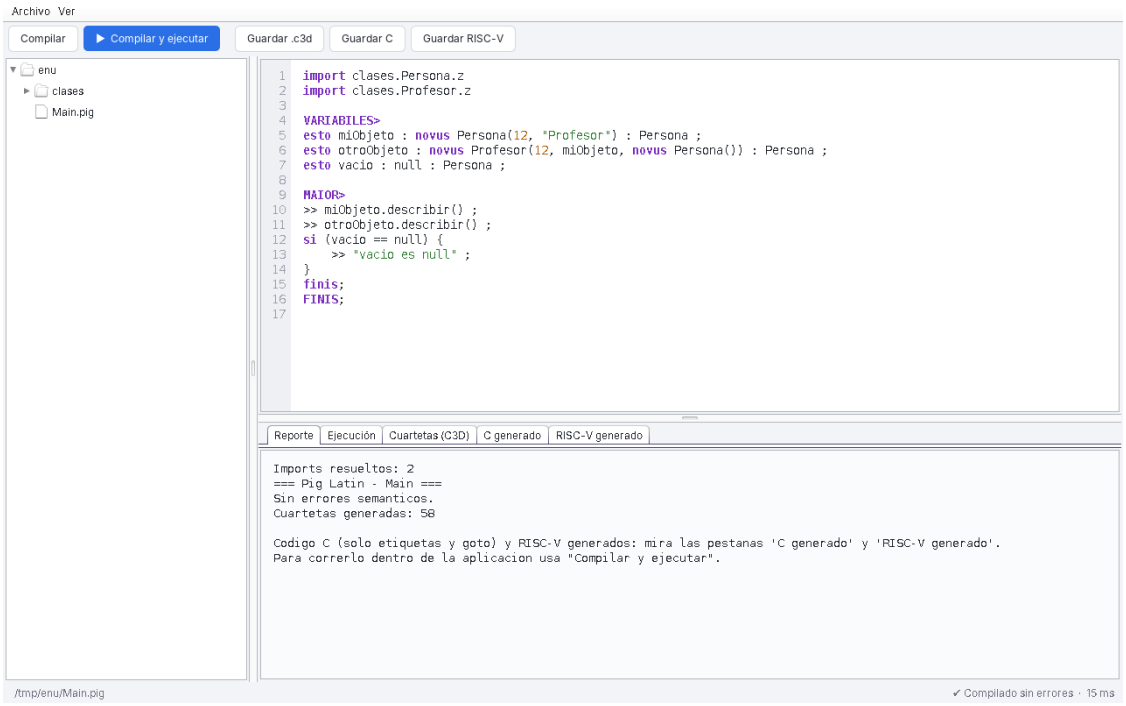


Figura 15. Tema claro

Ver > Cambiar tema alterna entre tema oscuro y claro.



3. Flujo de trabajo

- Escriba o abra un archivo .y, .z o .pig.
- Pulse Compilar. Si hay errores, aparecen con línea y columna en Reporte; corríjalos y repita.
- Pulse Compilar y ejecutar. Vaya a la pestaña Ejecución; cuando el programa pida datos, escríbalos en el campo inferior y presione Enter.
- Use Detener para interrumpir un programa en ejecución.
- Guarde los códigos generados con los botones de la barra.

Ejecución con RARS: guarde el .asm, ábralo en RARS, ensámblelo (F3) y ejecútelo (F5). La salida debe coincidir con la de la pestaña Ejecución.

4. Lenguaje Zetariano (.z)

Un archivo contiene una clase (la palabra public es opcional) y se llama igual que ella. Admite herencia con extends.

```
public class Guerrero extends Combatiente {
    private int fuerza;

    public Guerrero(String nombre, int fuerza) {
        super(nombre);
        this.fuerza = fuerza;
    }

    @Override
    public int atacar() {
        return fuerza * 2;
    }
```

```
}
```

4.1 Reglas principales

- Tipos: int, double, char, boolean, String, clases y arreglos (int[], int[][], ...).
- Modificadores de campos, constructores y métodos: public, private, protected o ninguno (default). protected: misma carpeta (paquete) o subclases; default: solo la misma carpeta; private: solo la propia clase.
- Paquetes: la carpeta desde la que se importa cada archivo (import carpeta.Clase.z) es su paquete.
- @Override es obligatorio al sobreescribir un método heredado, y es error usarlo si no hay nada que sobreescribir.
- Constructores sobrecargables; se invocan con new Clase(args). Se puede usar super(...) y super.metodo().
- Estructuras: if/else if/else, switch/case/default, for, while, do-while, break, continue, return, operador ternario.
- Arreglos: new int[3][4] y listas {1, 2, 3} para inicializar.
- Salida: println(x) con salto de línea y print(x) sin salto (también se acepta System.out.println y System.out.print).
- Entrada: x = read(); (o leer()) lee del teclado un int, double, char o String según el tipo del destino, sea variable, atributo o elemento.
- Operadores: + - * / %, comparaciones, && || ! (con cortocircuito), = += -= *= /= %=, ++ --.
- Arreglos: arr.length devuelve el tamaño de la primera dimensión (arr[i].length, el de la segunda).
- Un constructor puede delegar en otro de la misma clase con this(...) como primera sentencia.
- Strings: se pueden comparar con null (s == null).
- Cadenas: + concatena; == y != comparan su contenido.

5. Lenguajes Y? y Pig Latin

Y? usa estructuras y arreglos; imprimir(a, b) escribe sus argumentos en una línea. En Pig Latin la salida se hace con el operador >> encadenado (>> a >> b ; imprime una sola línea). En Pig Latin los objetos se declaran con su tipo explícito al final (esto nombre : novus Clase(args) : Clase ;) y pueden iniciarse en null (esto m : null : Forma ;).

```
import f.Forma.z

VARIABILES>
esto t : novus Taller(10) : Taller ;
esto m : null : Forma ;

MAIOR>
m = t.mayor();
>> "Mayor: " >> m.describir() ;
```

Los archivos de la carpeta ejemplos del proyecto muestran la sintaxis completa de cada lenguaje.

6. Mensajes de error

Situación	Mensaje / causa
Acceso a miembro private (o	Error semántico de visibilidad, con línea y columna.

Situación	Mensaje / causa
protected desde una clase no derivada)	
@Override sin método heredado	"tiene @Override pero no hereda..."
Método sobreescrito sin @Override	"sobreescribe un método heredado y debe llevar la anotación @Override".
Tipos incompatibles en asignación	Ver tabla de compatibilidad del manual técnico.
Anotación distinta de @Override	"única anotación reconocida: @Override".
Clase y archivo con nombres distintos	La clase debe llamarse como el archivo.

7. Notas y limitaciones

- Los booleanos se muestran como 1 (verdadero) y 0 (falso). Una cadena nula se muestra como null.
- Los import aceptan nombres con puntos, como import arena.Unidad.java.z (archivo Unidad.java.z dentro de la carpeta arena).
- Las estructuras se asignan por referencia.
- La lectura de cadenas toma una palabra (hasta el primer espacio) por línea.
- Un programa que no termina se corta por el límite de 2 000 millones de instrucciones; también se puede usar Detener.